

Arba Minch University

Faculty of Computing and Software Engineering

- Course Code ------- SENG1061
- Course Title -------- **Fundamental of Software Engineering**
- Degree Program --- SE 2nd Year Sem II

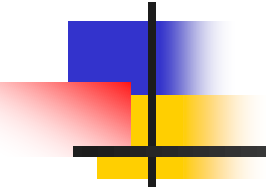


Arba Minch University

Faculty of Computing and Software Engineering

- Course Code ------- SENG1061
- Course Title -------- **Fundamental of Software Engineering**
- Degree Program --- SE 2nd Year Sem II

Fundamental of Software Engineering (SENG1061))



Chapter 06

Software Testing

Topics Covered

- Software Testing
- Software Testing Terminology
- Objective of Testing
- Good Testing Practice
- Types of Software Testing
- Software Testing Levels
- Software Testing Strategies

Objectives

- At the end of this session student will be able to:
  - Identify the type of software testing
  - Apply effective testing techniques to test a software system
  - Design specific and measurable test cases to ensure software requirements.
  - Critically reflect on the outcomes of software testing
  - Integrate testing processes within a continuous delivery model of software development

Observations about Testing

- "Testing is the process of executing a program with the intention of finding errors ".

  Myers

- "Testing can shows the presence of bugs but never their absence" Dijkstra

- According to **ANSI/IEEE 1059** standard, Testing can be defined as

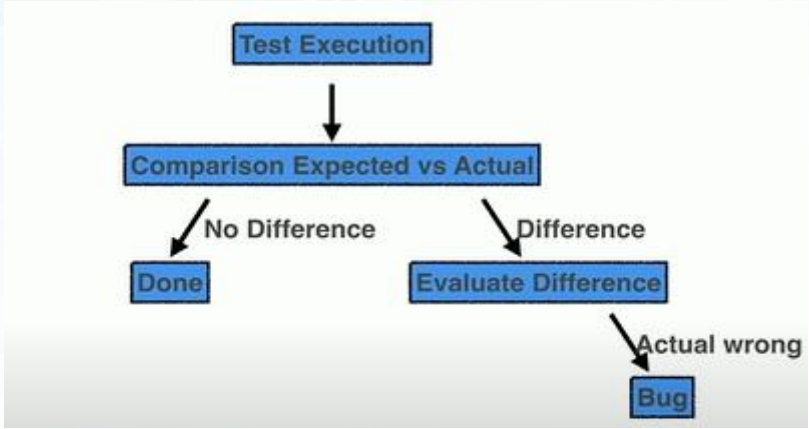
  A process of analyzing a software item to detect the differences between existing and required conditions (that is defects/errors/bugs) and to evaluate the features of the software item.

- Testing ensures that the software behaves and looks exactly like what is mentioned in the requirements specification document, so that when software is delivered to the client there are no arguments about the variation from the original requirements.

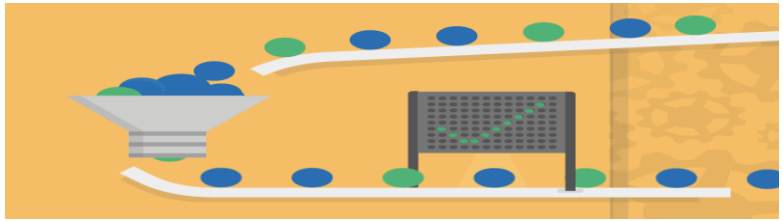
What is Testing?

- Verifying whether the application works as expected or not

- Used at key checkpoints in the overall process to determine whether objectives are being met







Software Testing Terminology

- **Test Case**
- A test case ,in a practical sense, is a test-related item which contains the following information:

1. **A set of test inputs**: These are data items received from an external source by the code under test. The external source can be hardware, software, or human.
2. **Execution conditions:** These are conditions required for running the test, for example, a certain state of a database, or a configuration of a hardware device.
3. **Expected outputs:** These are the specified results to be produced by the code under test.

Conti…

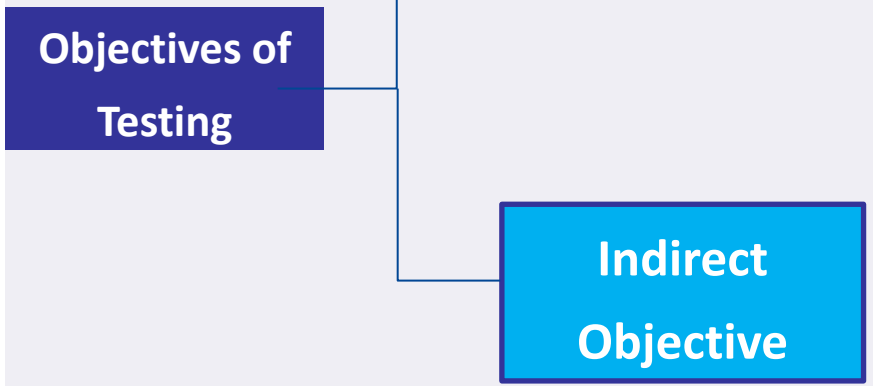
- **Test:**
  - A test is a group of related test cases and test procedures (steps needed to carry out a test)
- **Test Set:**

  A group of related tests is sometimes referred to as a test set.
- **Test Suite:**
  - Test Suite is a high level concept, grouping together hundreds or thousands of tests related by what they are intended to test
- **Test Log:** Defines the result of test cases in terms of passed/failed after execution of the test case on the application build.

Errors, Faults and Failures

- *Error:* A human action that produces an incorrect result.
- *It* refers to a missing or incorrect human action resulting in certain fault(s) being injected into a software.
- ***Fault:*** An incorrect step, process, or data definition in a computer program.
- *Fault* refers to an underlying condition within a software that causes certain failure(s) to occur;
- *Failure:* The **inability** of a system or component to perform its required functions within specified performance requirements.
- Failures, faults, and errors are collectively referred to as ***defects*** in literature
- Software problems or defects, are also commonly referred to as "Bugs/Issues"
- Errors ⟹ Faults ⟹ **Failures**

Objectives of Testing



- To identify and reveal as many errors as possible in the tested software
- To bring the tested software, after correction of the identified errors and retesting, to an acceptable level of quality
- To perform the required tests efficiently and effectively, within budgetary and scheduling limitations

- To compile a record of software errors for use in error prevention (by corrective and preventive actions)

Good Testing Practices

- A good test case is one that has a high probability of detecting an undiscovered defect, not one that shows that the program works correctly
- It is impossible to test your own program (Assign people)
- A necessary part of every test case is a description of the expected result
- Write test cases for valid as well as invalid input conditions
- Thoroughly inspect the results of each test
- As the number of detected defects in a piece of software increases, the probability of the existence of more undetected defects also increases

Conti…

- Ensure that testability is a key objective in your software design
- Never alter the program to make testing easier
- Testing, like almost every other activity, must start with objectives
- A good test is not redundant
  - Testing time is limited; one test should not serve the same purpose as another test
- A good test should be “best of breed”
  - Tests that have the highest likelihood of uncovering a whole class of errors should be used

Characteristics of a Testable software

- Operable
  - The better it works (i.e., better quality), the easier it is to test
- Observable
  - Incorrect output is easily identified; internal errors are automatically detected.
- Controllable
  - The states and variables of the software can be controlled directly by the tester
- Decomposable
  - The software is built from independent modules that can be tested independently

Conti…

- Simple
  - The program should exhibit functional, structural, and code simplicity
- Stable
  - Changes to the software during testing are infrequent and do not invalidate existing tests
- Understandable
  - The architectural design is well understood; documentation is available and organized

When to start testing

- An early start to testing reduces the cost, time to rework and error free software that is delivered to the client.
- However in Software Development Life Cycle (SDLC) testing can be started from the Requirements Gathering phase till the deployment of the software.
- However it also **depends on the development model that is being used**.
- For example, in Water fall model ,formal testing is conducted in the Testing phase,
- But in incremental model, testing is performed at the end of every increment/iteration and at the end the whole application is tested.

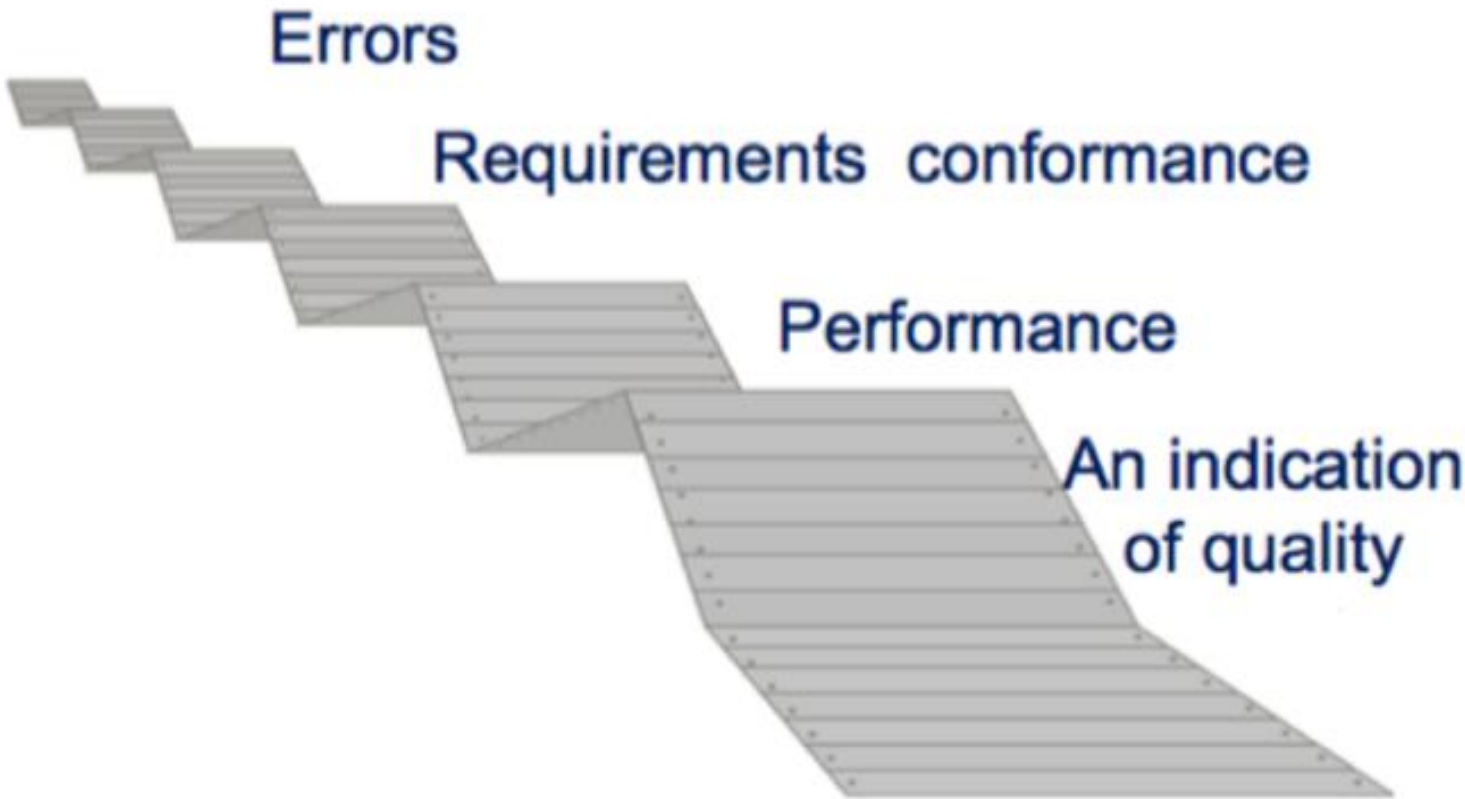
When to stop testing

- Unlike when to start testing, it is difficult to determine when to stop testing, as testing is a **never ending process** and no one can say that any software is 100% tested. Following are the aspects which should be considered to stop the testing:

  - Testing Deadlines.
  - Management decision.
  - Completion of test case execution.

- **Completion of Functional and code coverage to a certain point.**
- **Bug rate falls below a certain level and no high priority bugs are identified.**

What Testing Shows….?



Causes of Software Error

- **Requirement fault**
- **Defects in a product can come from any phase.** There could have been errors while gathering initial requirements.
- Defects have detrimental effects on software users, and software engineers work very hard to produce high-quality software with a low number of defects. But even under the best of development circumstances errors are made, resulting in defects being injected in the software during the phases of the software life cycle.

Conti…

- If, requirements are erroneously captured and the error is not detected until the product is delivered to the customer, the organization incurs extra expenses for
  - Performing a wrong design based on the wrong requirements
  - transforming the wrong design into wrong code during the coding phase
  - testing to make sure the product complies with the (wrong) requirement and
  - Releasing the product with the wrong functionality.



Conti…

- ***Education:*** The software engineer did not have the proper educational background to prepare the software artifact. For example, a software engineer who did not understand the precedence order of operators in a particular programming language could inject a defect in an equation that uses the operators for a calculation.
- ***Communication:*** The software engineer was not informed about something by a colleague. For example, if engineer 1 and engineer 2 are working on interfacing modules, and engineer 1 does not inform engineer 2 that a no error checking code will appear in the interfacing module he is developing, engineer 2 might make an incorrect assumption relating to the presence/absence of an error check, and a defect will result.
- ***Oversight:*** The software engineer omitted to do something. For example, a software engineer might omit an initialization statement.

Conti…

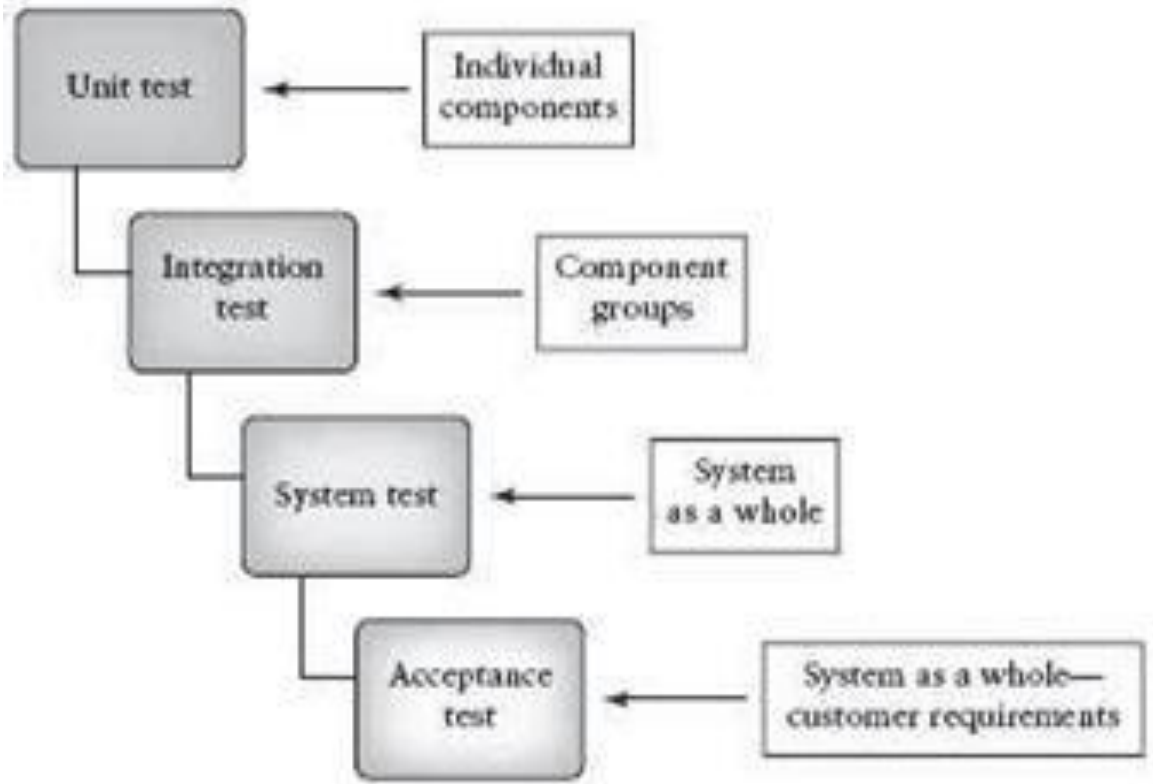
- ***Transcription:*** The software engineer knows what to do, but makes a mistake in doing it. A simple example is a variable name being misspelled when entering the code.
- ***Process:*** The process used by the software engineer misdirected her actions. For example, a development process that did not allow sufficient time for a detailed specification to be developed and reviewed could lead to specification defects.
- Logical design errors
- Non-compliance with documentation and coding instruction
- Shortcoming of the testing process
- Procedure error
- Documentation error

Manual Vs Automated Testing

- Manual testing is testing of the software where tests are executed manually by a QA Analyst. It is performed to discover bugs in software under development.
- In Manual testing, the tester checks all the essential features of the given application or software. In this process, the software testers execute the test cases and generate the test reports without the help of any automation software testing tools.
- Where as Automated Software Testing, testers write code/test scripts to automate test execution. Testers use appropriate automation tools to develop the test scripts and validate the software.

Levels of Software Testing

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing
  - Alpha Testing
  - Beta Testing



Unit Testing

- Unit testing is the method of testing various isolated software components separately.
- **Unit testing** is the technique of substantiating the smallest units in the software.
- These smallest components of the software are tested in isolation.
- A **driver** component is responsible for generating method calls for the components being tested.
- The component which utilizes a method is imitated as the **stub**. These stubs are the initial replacement for the misplaced methods.
- Focuses on the functional verification of the components.

Integration Testing

- The process of bringing together all of the modules that a program comprises for testing purposes.
- Integrate/combine the unit tested module one by one and test the behavior as a combined unit.



Integration Testing Strategies

- The two basic testing strategies are:
- To test the software in its entirely, once the completed package is available known as "big bang testing".
- To test the software piecemeal, in modules, as they are completed (unit test), then to test groups of tested modules integrated to the newly completed modules(integration testing). This process continues until all the package modules have been tested. Once this phase completed the entire package tested as a whole (system testing) the testing strategy is usually termed incremental testing.

big bang testing

- Big bang approach integrates all the modules in one go i.e. it does not go for integrating the modules one by one.
- It verifies if the system works as expected or not once integrated. If any issue is detected in the completely integrated module, then it becomes difficult to find out which module has caused the issue.
- Big bang approach is a time-consuming process of finding a module which has a defect itself as that would take time and once the defect is detected, fixing the same would cost high as the defect is detected at the later stage.

System Testing

- It is the technique in which the entire system is exercised with a series of the different tests.
- The **System Testing** verifies the behavior of the system entirely. It performs various tests in progression.
- However, these tests have the distinct intent and check whether all the system components are working correctly in an integrated way or not.
- The developers having the broad knowledge and visibility about the structure of the system usually performs the system testing.
- System testing relies on the high-level design specification and checks whether the system implementation satisfies the system requirement specification or not.

Type of System Testing

- **There are several forms of system testing among which few of them are:**
  - **Functional Testing**
  - **Recovery Testing**
  - **Regression Testing**
  - **Security Testing**
  - **Stress Testing**
  - **Performance Testing**

Conti…

- **Functional Testing**
  - Functional tests at the system level are used to ensure that the behavior of the system adheres to the requirements specification. All functional requirements for the system must be achievable by the system.
  - For example, if a personal finance system is required to allow users to set up accounts, add, modify, and delete entries in the accounts, and print reports, the function-based system and acceptance tests must ensure that the system can perform these tasks.
- In fact, the tests should focus on the following goals:
  - All types or classes of legal inputs must be accepted by the software
  - All classes of illegal inputs must be rejected (however, the system should remain available)
  - All possible classes of system output must exercised and examined
  - All functions must be exercised

Conti…

- **Recovery Testing**
- Recovery testing subjects a system to losses of resources in order to determine if it can recover properly from these losses.
- **Beizer advises that testers focus on the following areas during recovery testing:**
  - **Restart:** The current system state and transaction states are discarded. The most recent checkpoint record is retrieved and the system initialized to the states in the checkpoint record.
  - **Switchover:** The ability of the system to switch to a new processor must be tested.



Conti…

- **Regression Testing**
- Retesting of software that occurs when changes are made to ensure that the new version of the software has retained the capabilities of the old version and that no new defects have been introduced due to the changes.
- Regression testing can occur at any level of test, for example, when unit tests are run the unit may pass a number of these tests until one of the tests does reveal a defect.
- The unit is repaired and then retested with all the old test cases to ensure that the changes have not affected its functionality.

Conti…

- **Security Testing**:
- The security testing verifies the security mechanism and prevents the system
- Designing and testing software systems to ensure that they are safe and secure is a big issue facing software developers and test specialists.
- Recently, safety and security issues have taken on additional importance due to the proliferation of commercial applications for use on the Internet.
- Security testing evaluates system characteristics that relate to the availability, integrity, and confidentially of system data and services. Users/clients should be encouraged to make sure their security needs are clearly known at requirements time, so that security issues can be addressed by designers and testers.

Conti…

- **Stress Testing:**
- It is a testing technique in which the abnormal conditions of resources like quantity, frequency and volume are required.
- When a system is tested with a load that causes it to allocate its resources in maximum amounts, this is called stress testing. For example, if an operating system is required to handle 10 interrupts/second and the load causes 20 interrupts/second, the system is being stressed.
- The goal of stress test is to try to break the system; find the circumstances under which it will crash. This is sometimes called "**breaking the system**".
- Stress testing is important from the user/client point of view. When systems operate correctly under conditions of stress then clients have confidence that the software can perform as required.

Conti…

- **Performance Testing**
- It is an acceptance testing process used for testing the speed, response time, stability, reliability, scalability and resource usage of a software application under particular workload.
- The main purpose of performance testing is to identify and eliminate the performance bottlenecks in the software application.
- **The focus of Performance Testing is checking a software program's**
  - Speed – Determines whether the application responds quickly
  - Scalability – Determines maximum user load the software application can handle.
  - Stability – Determines if the application is stable under varying loads

Acceptance Testing

- Acceptance testing is a formal testing conducted to determine whether a system satisfies its acceptance criteria.
- The process whereby actual users test a completed information system, the end result of which is the users' acceptance of it.
- Acceptance tests are a very important milestone for the developers. At this time the clients will determine if the software meets their requirements.
- Contractual obligations can be satisfied if the client is satisfied with the software. Development organizations will often receive their final payment when acceptance tests have been passed

Beta Testing

- User testing of a completed information system using real data in the real user environment.
- Beta version of the software is released to a limited number of end-users of the product to obtain feedback on the product quality. Beta testing reduces product failure risks and provides increased quality of the product through customer validation.
- Beta testing is performed by Clients or End Users who are not employees of the organization

Alpha Testing

- Alpha testing is a type of **User Acceptance Testing** or internal acceptance testing conducted by the testing team at the **developer's site**.
- This is done to ensure that the application's business requirements are met before releasing it to the client or the end-user. It is carried out in the later phase of software development using both black-box and white-box testing techniques.
- Features of Alpha testing
  - It is performed by testing professionals, unlike beta testing which is performed by potential users.
  - It is performed at the developer's site.
  - It involves both black boxes as well as white box testing techniques.
  - It has a longer testing cycle as compared to beta testing

Conclusion

	Unit Testing	Integration	System	Acceptance
Why	To ensure the code is developed correctly	To ensure the ties between the system component functions as required	To ensure the whole system worked when integrated	To ensure customers and end user expectation met
Who	Developer/Technical Architects	Developer/Technical Architects	SDET/Manual QA, BA/ Product owner	Developer, SDET/Manual QA,/Product owner, end user
What	All new code	New web service components, controllers	Scenario testing User flow	Verifying Acceptance testing
When	As soon as a new code is written	As soon as a new component is written	When the product is complete	When the product is shipped
Where	Local Dev+ Continuous integration	Local Dev+ CI (Part of the build)	Staging Environment	CI/ Test Environment
How	Automated, Junit, TestNG	Automated, Soup UI	Automated (web driver)	Automated(Cucumber)

Verification and Validation

- **Verification in Software Testing** is a process of checking documents, design, code, and program in order to check if the software has been built according to the requirements or not.
- The main goal of verification process is to ensure quality of software application, design, architecture etc. The verification process involves activities like reviews, walk-throughs and inspection.
- **Validation in Software Engineering** is a dynamic mechanism of testing and validating if the software product actually meets the exact needs of the customer or not. The validation process involves activities like unit testing, integration testing, system testing and user acceptance testing.

Conti…

- **KEY DIFFERENCE**
  - Verification process includes checking of documents, design, code and program whereas Validation process includes testing and validation of the actual product.
  - Verification uses methods like reviews, walkthroughs, inspections and desk-checking whereas Validation uses methods like black box testing, white box testing and non-functional testing.
  - Verification checks whether the software confirms a specification whereas Validation checks whether the software meets the requirements and expectations.
  - Verification finds the bugs early in the development cycle whereas Validation finds the bugs that verification can not catch.

END

Q?